Playing with OR

You are given an array A containing N integers, and an integer K ($1 \le K \le N$). Find the number of *subarrays* of A with length K whose bitwise OR is odd.

Note:

- A subarray of A is a contiguous segment of elements of A.
- For example, if A=[1,3,2], then it has 6 non-empty subarrays: [1],[3],[2],[1,3],[3,2],[1,3,2].
- In particular, [1,2] is *not* a subarray of A.

Input Format

- The first line of input will contain a single integer T, denoting the number of test cases.
- Each test case consists of two lines of input.
  - The first line of each test case contains two space-separated integers N and K — the length of the array and the subarray size you have to check, respectively.
  - The second line of each test contains N space-separated integers A1, A2,…,AN — the elements of the array.

Output Format

For each test case, output on a new line the number of length-K subarrays of A whose bitwise OR is odd.

Constraints

- $1 \le T \le 10^5$
- $1 \le K \le N \le 5 \cdot 10^5$
- $1 \le A_i \le 10^9$
- The sum of N across all tests doesn't exceed $5 \cdot 10^5$.

Sample 1:

Input
Output

```
2
5 2
5 7 13 4 6
4 3
2 6 7 4

3
2
```

Test case 2: There are two subarrays of length three, both of them have odd bitwise OR.

APPROACH 1 — CHECK EVERY SUBARRAY DIRECTLY

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int T = sc.nextInt();
      while (T > 0) {
         int N = sc.nextInt();
         int K = sc.nextInt();
         int[] A = new int[N];
         for (int i = 0; i < N; i++)
                  A[i] = sc.nextInt();
         int answer = 0;
         for (int i = 0; i <= N - K; i++) {
            int orValue = 0;
            for (int j = i; j < i + K; j++)
                  orValue |= A[j];
             if ((orValue & 1) == 1)
                  answer++;
          }
         System.out.println(answer);
         T--;
      }
   }
}
```

Explanation:

For the array [5 7 13 4 6] and k =2,

There are four subarrays of length K=2.

[5, 7], with bitwise OR equal to 7.

[7, 13], with bitwise OR equal to 15.

[13, 4], with bitwise OR equal to 13.

[4, 6], with bitwise OR equal to 6.

Three of them are odd, so the answer is 3.

Complexity

There are approximately N-K+1 subarrays, and each takes K operations.

Time: O(N × K)

Space: O(N) for the array.

This can be too slow because N can be 5×10^5.

APPROACH 2 — CHECK WHETHER EACH WINDOW CONTAINS AN ODD NUMBER

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int T = sc.nextInt();
      while (T-- > 0) {
         int N = sc.nextInt();
         int K = sc.nextInt();
         int[] A = new int[N];
         for (int i = 0; i < N; i++) {
            A[i] = sc.nextInt();
         }
         int oddCount = 0;
         for (int i = 0; i < K; i++) {
            if ((A[i] & 1) == 1) {
               oddCount++;
            }
         }
         int answer = 0;
         if (oddCount > 0) {
            answer++;
         }
         for (int i = K; i < N; i++) {
            if ((A[i] & 1) == 1) {
               oddCount++;
            }
            if ((A[i - K] & 1) == 1) {
               oddCount--;
            }
            if (oddCount > 0) {
               answer++;
            }
         }
         System.out.println(answer);
      }
   }
}
```

Complexity

Each element is added once and removed once.

Time: O(N)

Space: O(N)

Explanation:
One importatnt observation is:
ORValue of sub array is odd if atleast one element is odd.

We can simply check whether each length-K window contains an odd element.

For example:

A = [5, 7, 13, 4, 6]
O O O E E

Window size = 2

[5,7] → contains odd → YES
[7,13] → contains odd → YES
[13,4] → contains odd → YES
[4,6] → no odd → NO

We can use a sliding window.

Maintain: oddCount = number of odd elements in current window

APPROACH 3 — EVEN SIMPLER: PREFIX COUNT OF ODD NUMBERS

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int T = sc.nextInt();
      while (T-- > 0) {
         int N = sc.nextInt();
         int K = sc.nextInt();
         int[] prefix = new int[N + 1];
         for (int i = 0; i < N; i++) {
            int x = sc.nextInt();
            prefix[i + 1] = prefix[i];
            if ((x & 1) == 1) {
               prefix[i + 1]++;
            }
         }
         int answer = 0;
         for (int L = 0; L <= N - K; L++) {
            int R = L + K;
            int oddCount = prefix[R] - prefix[L];
            if (oddCount > 0) {
               answer++;
            }
         }
         System.out.println(answer);
      }
   }
}
```

Complexity

Time: O(N)

Space: O(N)

Explanation:
Create a prefix array.
Let: prefix[i] = number of odd elements among A[0 ... i-1]

For example:
A = [5, 7, 13, 4, 6]
odd? 1 1 1 0 0

prefix = [0, 1, 2, 3, 3, 3]

For a window [L ... R], number of odd elements is:

prefix[R + 1] - prefix[L]

If this is greater than 0, **the OR is odd.**

APPROACH 4 — STORE ONLY WHETHER EACH ELEMENT IS ODD

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int N = sc.nextInt();
            int K = sc.nextInt();
            int[] odd = new int[N];
            for (int i = 0; i < N; i++) {
                int x = sc.nextInt();
                odd[i] = x & 1;
            }
            int count = 0;
            int answer = 0;
            for (int i = 0; i < K; i++) {
                count += odd[i];
            }
            if (count > 0) {
                answer++;
            }
            for (int i = K; i < N; i++) {
                count += odd[i];
                count -= odd[i - K];
                if (count > 0) {
                    answer++;
                }
            }
            System.out.println(answer);
        }
    }
}
```

Explanation:
We don't even need the actual values after reading them.

Convert each element to:

odd → 1
even → 0

Then the problem becomes:

Count length-K subarrays containing at least one 1.

We can use a sliding window.

Time: O(N)

Space: O(N)

COMPARISON OF APPROACHES

Approach	Main idea	Time	Space
1. Direct OR	Calculate OR for every window	O(NK)	O(N)
2. Sliding window	Maintain count of odd elements	O(N)	O(N)
3. Prefix count	Prefix number of odd elements	O(N)	O(N)
4. Parity array + sliding window	Store only odd/even	O(N)	O(N)

INTERVIEW QUESTIONS VERY SIMILAR TO Playing with OR

No	Interview question	Company	Year	Job Role	Platform
1	**Count distinct Bitwise OR of all subarrays**	Deloitte	2021	SDE - 1	LeetCode 898 GeeksForGeeks
		Jio Platforms	2022	SWE	
		Mercer Mettl	2021	SQE	
		Codenation	2020	SDE Intern	
		Google	2020	SDE Intern	
		Nokia	2022	SDE - 1	
2	**Remove Maximum length subarray while preserving maximum OR**	Interview Question	2024	Not Specified	LeetCode Interview Questions
3	**Find XOR of ANDs of all subarrays of size ≤ K**	Interview Question	2021	Ode 2 Code Mi coding contest.	CodeForces
4	**Bitwise OR of all subsequence sums**	Zomato	2025	SDE - 1	LeetCode 2505
5	**Count subarrays with exactly K odd numbers**	Infosys	2025	SDE-1	LeetCode 1248